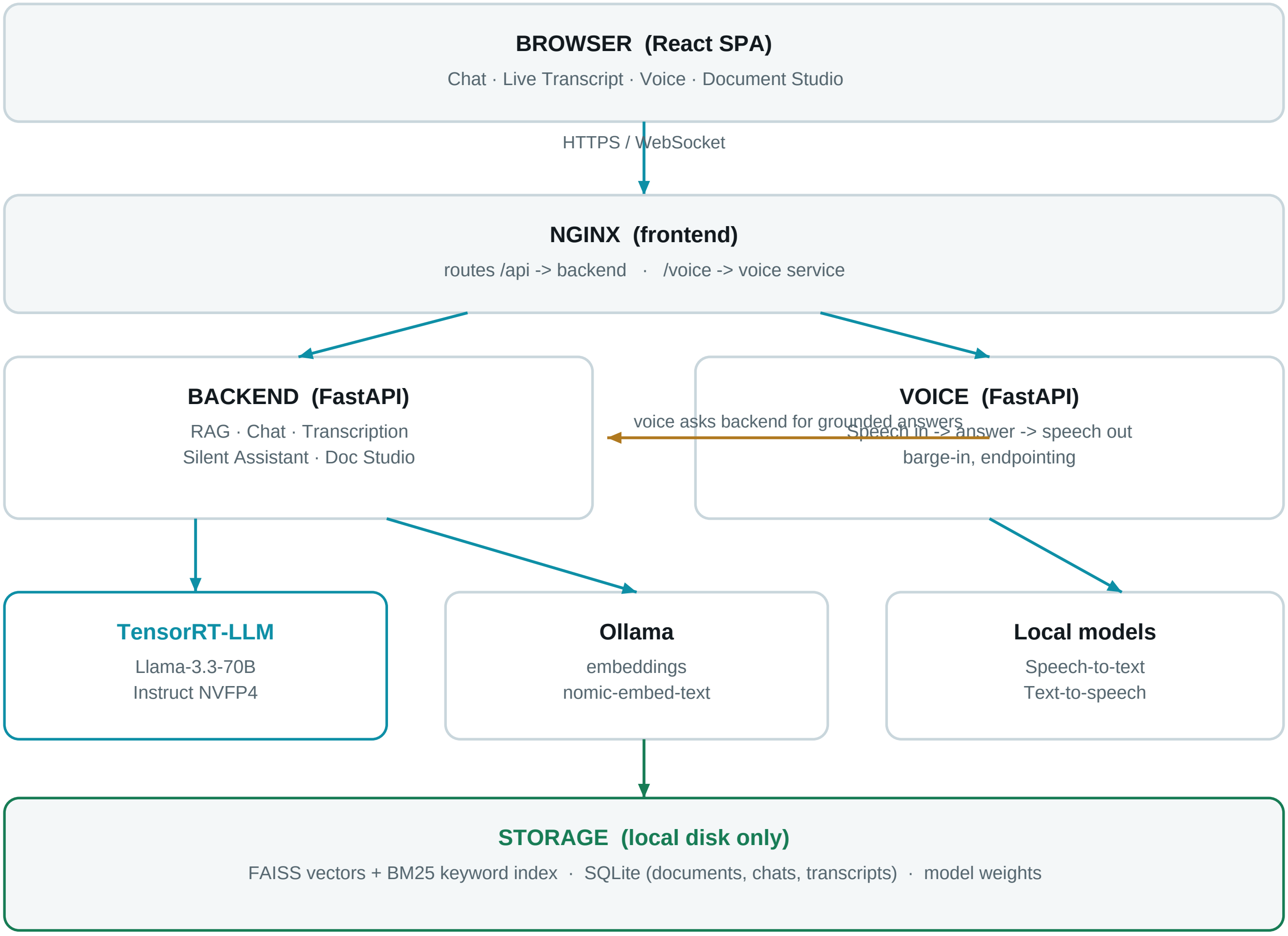
How EchoMind Works

End-to-end architecture in one read

EchoMind is a private AI assistant that answers from your own documents.
It runs entirely on one machine. No cloud, no external API calls, no data leaves the box.
Five Docker services. One web app. Everything below happens on-premise.

System architecture



Nothing in this diagram reaches the internet at runtime.

How a Question Becomes an Answer

The retrieval pipeline, step by step

Step 1 — Getting documents in (ingestion)

- You upload a PDF, Word file or PowerPoint.
- The text is extracted and cut into passages of roughly 450 words.
- Each passage is turned into a list of numbers (an embedding) that captures its meaning.
- Passages are filed in two indexes: one for meaning (FAISS), one for exact words (BM25).
- Every passage is tagged with its tenant namespace, so customers stay separated.

Step 2 — Answering a question

1	**Route**	Is this small talk or a real question? Greetings never touch the knowledge base.
2	**Search**	Two searches at once: by meaning and by exact keywords.
3	**Merge**	Both result lists are fused and ranked (weighted RRF).
4	**Re-read**	A cross-encoder re-reads the top 25 against your question, keeps the best 15.
5	**Gate**	If nothing is truly relevant, the context is dropped rather than forced in.
6	**Assemble**	Best passages are packed with source labels, fenced as untrusted data.
7	**Answer**	The LLM writes the answer from that evidence only, with citations.

Why answers can be trusted

- Every claim carries a citation: document, section and page.
- Retrieved text is fenced as data, never as instructions — a document cannot hijack the assistant.
- If the documents do not contain the answer, the assistant says so instead of guessing.
- Tenant namespaces are enforced inside the search itself, not filtered afterwards.

Voice, and Where the Code Lives

The speech loop and a file map

How the voice assistant works

Mic	→	Detect	→	Listen	→	Decide	→	Transcribe	→	Answer	→	Speak
20 ms audio frames		is this speech?		live captions (streaming model)		have they finished?		accurate final transcript		same RAG as the chat		phrase-by-phrase audio out

- It starts talking within about half a second — a short lead phrase covers the thinking time.
- It knows when you have finished: a complete sentence gets a fast reply, a trailing one waits longer.
- Talk over it and it stops immediately, then listens.
- The spoken answer comes from the same documents as the typed answer.

Where things live in the code

`backend/app/rag/`	Retrieval, ranking, prompts, personas — the core of the answer engine
`backend/app/rag/index.py`	FAISS + BM25 indexes, tenant namespace enforcement
`backend/app/rag/advanced.py`	The answer pipeline: route, search, rerank, assemble, generate
`backend/app/transcribe/`	Live transcription and the Silent Assistant fact-checker
`voice/app/session.py`	The whole voice loop: listening, endpointing, barge-in, speaking
`frontend/`	React app; packs.ts maps each subdomain to its own tenant
`docker-compose.yml`	The five services, their models and GPU assignment
`eval/`	Automated quality suite — 52 golden questions, run before every release

Deployment

- One NVIDIA DGX Spark: 128 GB unified memory, GB10 Grace-Blackwell.
- Five containers started with a single command.
- Model weights are baked in — the system runs with the network cable unplugged.
- Optional public access via Cloudflare Tunnel, gated by a login wall.